π_{RL}

基于 Flow Matching 的 VLA 强化学习

Flow-SDE、Flow-Noise 与 PPO 的数学原理和实现

作者：Yang kunpeng

论文：arXiv:2510.25889

目录

第一章 Flow-SDE：从 ODE 到随机策略 **1**

1.1 确定性 Flow 为什么难以直接用于强化学习 1

1.2 连续性方程：ODE 如何推动概率密度变化 2

1.3 Fokker–Planck 方程：随机扩散如何改变密度 3

1.4 匹配密度演化与 ODE-to-SDE 4

1.5 高斯转移、对数概率与两层 MDP 5

1.6 官方采样器：均值、标准差与对数概率 7

1.7 官方 Rollout：随机转移如何进入训练数据 10

第二章 Flow-Noise：通过学习噪声构造随机策略 **13**

2.1 从确定性 Euler 更新到高斯转移 13

2.2 为什么整条去噪轨迹的对数概率可计算 14

2.3 官方噪声网络与转移计算 15

2.4 Flow-Noise 的训练与推理 17

第三章 PPO 与 VLA 强化学习训练 **19**

3.1 环境交互、动作块与奖励 19

3.2 从对数概率到 PPO 概率比 19

3.3 Critic 与 GAE 优势估计 20

3.4 在线训练循环与参数更新 21

3.5 推理：从观测到机器人动作 24

第 1 章

Flow-SDE：从 ODE 到随机策略

1.1 确定性 Flow 为什么难以直接用于强化学习

Flow-based VLA 根据机器人观测生成一段连续动作，而不是逐个输出离散动作 token。令环境时刻为 t，观测为 $\mathbf{o}_t$，动作块为

$$\mathbf{A}_t = [\mathbf{a}_{t,0}, \mathbf{a}_{t,1}, \ldots, \mathbf{a}_{t,H-1}].$$

观测包含相机图像、语言指令和机器人本体状态。VLA 的视觉语言模块提取条件特征，Action Expert 通过 Flow Matching 学到条件速度场

$$\mathbf{v}_\theta(\mathbf{A}^\tau, \mathbf{o}_t),$$

再将初始噪声逐步变换为动作块。

这里有两个不同的时间变量。t 是机器人与环境交互的离散时刻；$\tau \in [0, 1]$ 是生成动作块时使用的 Flow 时间。为避免混淆，后文省略动作块的环境下标 t，并将 $\mathbf{A}^\tau$ 简写为 $\mathbf{A}^\tau$。

原始 Flow 采样可写成常微分方程（ODE）：

$$\frac{\mathrm{d}\mathbf{A}^\tau}{\mathrm{d}\tau} = \mathbf{v}^\tau, \qquad \mathbf{v}^\tau = \mathbf{v}_\theta(\mathbf{A}^\tau, \mathbf{o}).$$

用步长 $\delta = 1/K$ 作前向 Euler 离散化：

$$\mathbf{A}^{\tau+\delta} = \mathbf{A}^\tau + \mathbf{v}^\tau \delta.$$

给定当前状态和初始噪声后，每一步都是确定的。强化学习却需要比较新旧策略对已采样动作的概率，构造策略梯度或 PPO 的概率比。直接计算由有限次 ODE 积分得到的最终动作精确似然并不方便；少量去噪步下，基于连续流密度变化的数值似然也可能不够准确。

Flow-SDE 的做法是在生成过程中引入随机转移，使每个去噪步骤对应可计算的高斯概率。随机性服务于训练时的探索和策略概率计算；强化学习训练结束后，部署仍可使用确定性 ODE 生成动作。

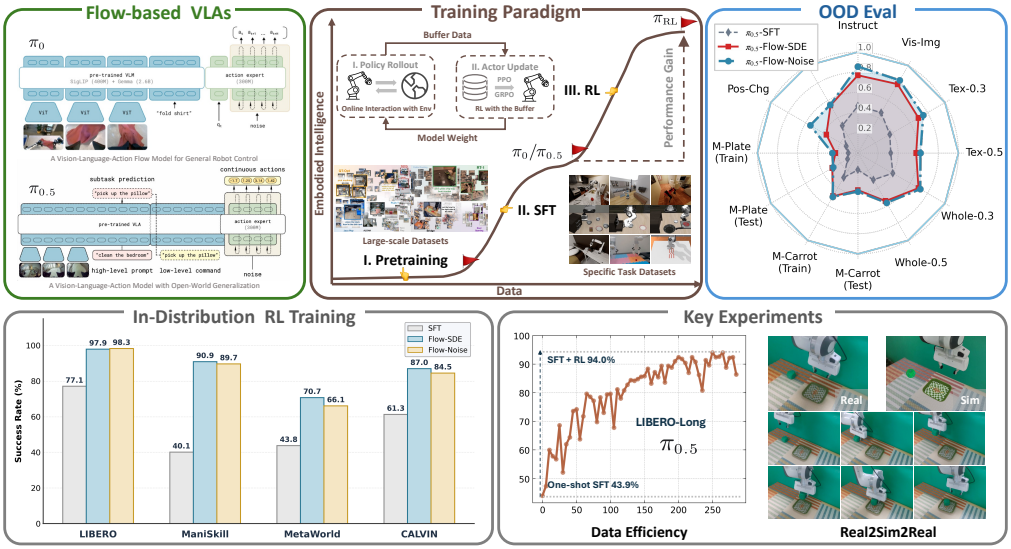


图 1.1: π_{RL} 的总体框架。Flow-Noise 与 Flow-SDE 为 Flow-based VLA 提供可用于在线强化学习的随机转移机制。

1.2 连续性方程：ODE 如何推动概率密度变化

设随机状态 $\mathbf{X}_\tau$ 按照 ODE 运动：

$$\frac{\mathrm{d}\mathbf{X}_\tau}{\mathrm{d}\tau} = \mathbf{v}_\tau(\mathbf{X}_\tau),$$

其概率密度记为 $q_\tau(\mathbf{x})$。速度场不仅改变单个样本的位置，也会使样本群体的密度随时间变化。概率质量守恒意味着：一个区域内部概率的变化，只能来自穿过该区域边界的概率流。

取空间区域 Ω，区域内的概率质量为

$$P(\mathbf{X}_\tau \in \Omega) = \int_\Omega q_\tau(\mathbf{x})\,\mathrm{d}\mathbf{x}.$$

边界上的概率流密度是 $q_\tau \mathbf{v}_\tau$。令 $\mathbf{n}$ 为区域边界的外法向量，散度定理给出

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}\tau}\int_\Omega q_\tau(\mathbf{x})\,\mathrm{d}\mathbf{x} &= -\int_{\partial\Omega} q_\tau(\mathbf{x})\mathbf{v}_\tau(\mathbf{x})\cdot\mathbf{n}\,\mathrm{d}S \\ &= -\int_\Omega \nabla\cdot(q_\tau\mathbf{v}_\tau)\,\mathrm{d}\mathbf{x}.\end{aligned}$$

由于区域 Ω 任意，得到连续性方程：

$$\frac{\partial q_\tau}{\partial \tau} = -\nabla\cdot(q_\tau\mathbf{v}_\tau).$$

其中 $\nabla\cdot$ 是散度算子，衡量向量场在局部是净流出还是净流入。若某处速度场的散度为正，附近样本倾向于向外分散；若为负，样本倾向于汇聚。连续性方程表达的是确定性流动下的概率守恒。

1.3 Fokker–Planck 方程：随机扩散如何改变密度

将确定性 ODE 改成随机微分方程（SDE）：

$$\mathrm{d}\mathbf{X}_\tau = \mathbf{b}_\tau(\mathbf{X}_\tau)\,\mathrm{d}\tau + g(\tau)\,\mathrm{d}\mathbf{W}_\tau.$$

$\mathbf{b}_\tau$ 是漂移项，决定平均运动方向；$g(\tau)$ 控制噪声强度；$\mathbf{W}_\tau$ 是标准 Wiener 过程，$\mathrm{d}\mathbf{W}_\tau$ 是均值为零、协方差为 $\mathbf{I}\mathrm{d}\tau$ 的随机增量。

对任意足够光滑的函数 f，Itô 公式给出

$$\mathrm{d}f(\mathbf{X}_\tau) = \nabla f^{\mathsf{T}}\mathbf{b}_\tau\,\mathrm{d}\tau + g(\tau)\nabla f^{\mathsf{T}}\mathrm{d}\mathbf{W}_\tau + \frac{1}{2}g^2(\tau)\Delta f\,\mathrm{d}\tau$$

这里假设 f 不显含时间；若 f 还显含 τ，右侧需增加 $\partial f/\partial\tau\,\mathrm{d}\tau$。取期望后，随机积分项的期望为零。再对空间变量分部积分，就得到密度演化方程

$$\frac{\partial q_\tau}{\partial \tau} = -\nabla\cdot(\mathbf{b}_\tau q_\tau) + \frac{g^2(\tau)}{2}\Delta q_\tau.$$

这就是各向同性扩散下的 Fokker–Planck 方程。与连续性方程相比，它多出拉普拉斯项 $\frac{g^2}{2}\Delta q_\tau$，该项描述随机扩散对概率密度的平滑作用。这里的 $\Delta = \nabla\cdot\nabla$ 是拉普拉斯算子。

1.4 匹配密度演化与 ODE-to-SDE

若要求 ODE 与 SDE 在同一时间方向上具有相同的边缘密度 q_τ，则两种密度方程必须一致：

$$-\nabla \cdot (q_\tau \mathbf{v}_\tau) = -\nabla \cdot (q_\tau \mathbf{b}_\tau) + \frac{g^2(\tau)}{2} \Delta q_\tau.$$

利用

$$\nabla q_\tau = q_\tau \nabla \log q_\tau, \qquad \nabla \cdot (q_\tau \nabla \log q_\tau) = \Delta q_\tau,$$

可得在通常的同向时间约定下，一个满足该等式的漂移为

$$\mathbf{b}_\tau = \mathbf{v}_\tau + \frac{g^2(\tau)}{2} \nabla \log q_\tau.$$

因此，扩散项并不是简单叠加在原 ODE 上。为了维持指定的边缘密度演化，漂移项必须同时作相应修正。Score 定义为

$$\mathbf{s}_\tau(\mathbf{x}) = \nabla_{\mathbf{x}} \log q_\tau(\mathbf{x}),$$

它指向概率密度上升最快的局部方向。

论文符号与官方实现的时间方向。论文用 τ 表示从噪声到动作的递增时间：$\mathbf{A}^\tau = \tau \mathbf{A} + (1-\tau)\boldsymbol{\epsilon}$，因此 $\tau = 0$ 对应噪声，$\tau = 1$ 对应动作。RLinf 当前官方实现中的 `get_timesteps` 则返回从 1 递减到 0 的网格，并从高斯噪声开始迭代到动作。为区分两者，以下用 u 表示实现中的时间变量，满足 $u = 1 - \tau$。论文中的速度方向、Score 修正符号和噪声调度不能在不变换时间变量的情况下逐项照抄到实现中。

论文的连续时间推导给出漂移修正与 Score 的关系；官方采样器则把它写成离散的均值权重。其实现采用

$$\sigma_i = a \sqrt{\frac{u_i}{1 - u_i}}, \qquad \Delta_i = u_i - u_{i+1} > 0,$$

并在首个时间点对分母作数值保护。这里的 σ_i 是单步转移的标准差系数，单步标准差还要乘以 $\sqrt{\Delta_i}$。后文直接引用官方采样器的均值和标准差计算，避免把论文的递增 τ 公式与实现中的递减 u 网格混用。

1.5 高斯转移、对数概率与两层 MDP

在论文的递增 τ 符号约定下，用 Euler–Maruyama 方法离散 SDE。令 δ 为去噪步长，则单步转移可写为

$$\mathbf{A}^{\tau+\delta} = \boldsymbol{\mu}_\tau + \sigma_\tau \sqrt{\delta}\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

其中

$$\boldsymbol{\mu}_\tau = \mathbf{A}^\tau + \left[\mathbf{v}^\tau + \frac{\sigma_\tau^2}{2\tau}\left(\mathbf{A}^\tau + (1-\tau)\mathbf{v}^\tau\right)\right]\delta, \qquad \boldsymbol{\Sigma}_\tau = \sigma_\tau^2 \delta \mathbf{I}.$$

因此每一步的条件分布是高斯分布：

$$p(\mathbf{A}^{\tau+\delta} \mid \mathbf{A}^\tau, \mathbf{o}) = \mathcal{N}(\boldsymbol{\mu}_\tau, \boldsymbol{\Sigma}_\tau).$$

给定样本后，可以计算这一步的高斯对数概率。对一个 D 维状态，通用形式为

$$\log p(\mathbf{x}) = -\frac{1}{2}\left[D\log(2\pi) + \log|\boldsymbol{\Sigma}| + (\mathbf{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right].$$

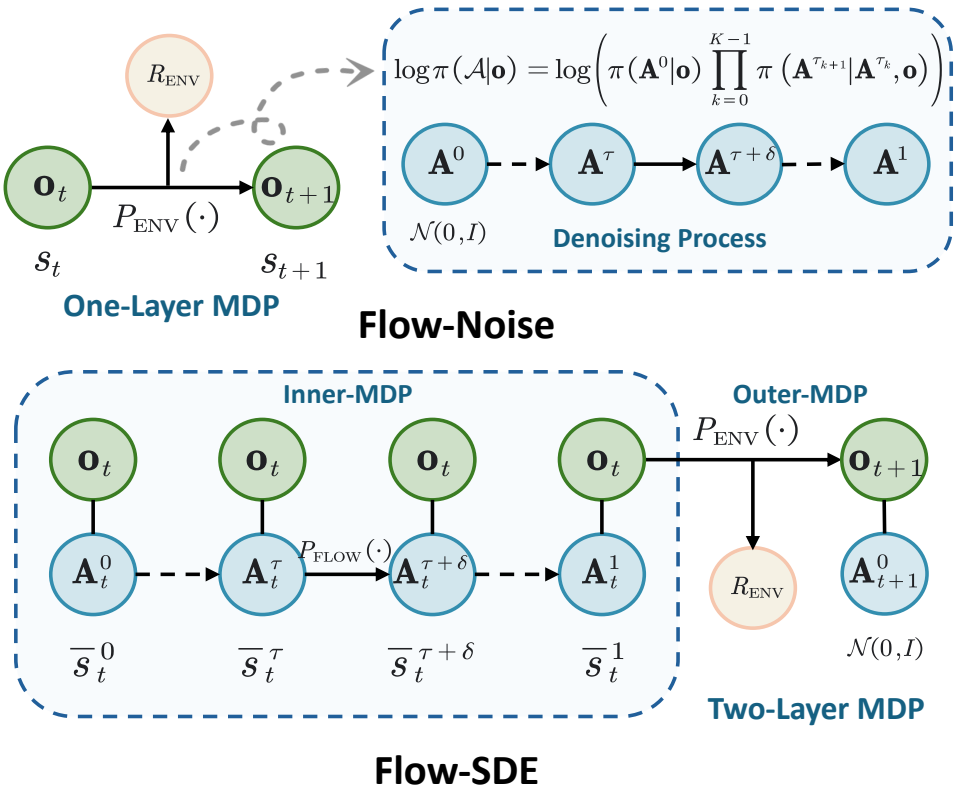


图 1.2: Flow-Noise 的单层 MDP 与 Flow-SDE 的两层 MDP。Flow-SDE 将内部去噪状态转移和外部机器人环境交互显式区分。

Flow-SDE 的状态同时包含环境观测和当前去噪状态：

$$\bar{s}_t^{\tau} = (\mathbf{o}_t, \mathbf{A}_t^{\tau}).$$

当 $\tau < 1$ 时，策略动作是下一步去噪状态 $\mathbf{A}_t^{\tau+\delta}$，观测 $\mathbf{o}_t$ 保持不变，内部奖励为零。当去噪到达 $\tau = 1$ 时，最终动作块 $\mathbf{A}_t^1$ 与环境交互，得到环境奖励和新观测 $\mathbf{o}_{t+1}$，随后从初始噪声分布开始下一个动作块。于是内部去噪形成内层 MDP，机器人与环境交互形成外层 MDP。

标准两层形式的有效决策链较长。论文采用混合 ODE–SDE rollout：每个环境时刻随机选择一个去噪时间进行随机 SDE 转移，其余去噪步骤由环境包装器按确定性 ODE 执行。这样减少了需要由策略显式建模和记录的随机决策步数，同时保留随机转移与环境奖励之间的联系。

1.6 官方采样器：均值、标准差与对数概率

RLinf 将单步转移计算集中在官方文件 `rl_sampler.py` 的 `sample_mean_var` 与 `gaussian_logprob` 中。采样器使用递减时间网格，先根据当前状态和速度预测两个端点，再通过权重组合得到下一状态的均值。Flow-SDE 分支在权重中加入漂移修正，并令单步标准差为 $\sigma_i\sqrt{\Delta_i}$；Flow-Noise 分支保留 ODE 均值，直接使用噪声网络输出的标准差。

Listing 1.1: RLinf 官方采样器节选：时间网格、Flow-SDE/Flow-Noise 转移参数与高斯对数概率。

```python
# Copyright 2026 The RLinf Authors.
# Licensed under the Apache License, Version 2.0.
# Source: rlinf/models/embodiment/openpi/sampling/rl_sampler.py
# https://github.com/RLinf/RLinf/blob/main/rlinf/models/embodiment/openpi/sampling/
    ↪ rl_sampler.py
#
# Curated excerpt for the book: retains flow_ode, flow_sde, flow_noise and
# Gaussian log-probability code; unrelated flow_cps branch is omitted.

from __future__ import annotations

import math
import torch


def get_timesteps(num_steps: int, device: torch.device | str) -> torch.Tensor:
    """Return [1, (N-1)/N, ..., 1/N, 0] of length num_steps + 1."""
    timesteps = torch.linspace(1.0, 1.0 / num_steps, num_steps, device=device)
    return torch.cat([timesteps, torch.zeros(1, device=device)])


def sample_mean_var(
    x_t: torch.Tensor,
    v_t: torch.Tensor,
    idx: torch.Tensor,
    *,
    noise_method: str,
    noise_level: float,
```

```
    num_steps: int,
    noise_std: torch.Tensor | None = None,
) -> tuple[torch.Tensor, torch.Tensor]:
    """Official RLinf transition parameterization, excerpted for the book."""
    device = x_t.device
    timesteps = get_timesteps(num_steps, device).to(x_t.dtype)
    t_input = timesteps[idx][:, None, None].expand_as(x_t)
    delta = (timesteps[idx] - timesteps[idx + 1])[:, None, None].expand_as(x_t)

    x0_pred = x_t - v_t * t_input
    x1_pred = x_t + v_t * (1 - t_input)

    if noise_method == "flow_ode":
        x0_weight = 1 - (t_input - delta)
        x1_weight = t_input - delta
        x_t_std = torch.zeros_like(t_input)

    elif noise_method == "flow_sde":
        denom_timesteps = torch.where(timesteps == 1, timesteps[1], timesteps)
        sigma_ratio = timesteps / (1 - denom_timesteps)
        sigmas = noise_level * torch.sqrt(sigma_ratio)[:-1]
        sigma_i = sigmas[idx][:, None, None].expand_as(x_t)

        x0_weight = torch.ones_like(t_input) - (t_input - delta)
        x1_weight = (t_input - delta) - sigma_i * sigma_i * delta / (2 * t_input)
        x_t_std = torch.sqrt(delta) * sigma_i

    elif noise_method == "flow_noise":
        if noise_std is None:
            raise ValueError(
                "noise_method='flow_noise' requires noise_std from ExploreNoiseNet."
            )
        x0_weight = 1 - (t_input - delta)
        x1_weight = t_input - delta
        x_t_std = noise_std.to(dtype=x_t.dtype)

    else:
        raise NotImplementedError(
            f"noise_method={noise_method!r} is not implemented. "
            "Supported excerpt: 'flow_ode', 'flow_sde', 'flow_noise'."
```

```
        )

    x_t_mean = x0_pred * x0_weight + x1_pred * x1_weight
    return x_t_mean, x_t_std


def gaussian_logprob(
    sample: torch.Tensor, mu: torch.Tensor, sigma: torch.Tensor
) -> torch.Tensor:
    """Per-element Gaussian log-probability; zero wherever sigma == 0."""
    mask = sigma == 0
    sigma_safe = torch.where(mask, torch.ones_like(sigma), sigma)
    log_two_pi = math.log(2 * math.pi)
    log_prob = (
        -torch.log(sigma_safe)
        - 0.5 * log_two_pi
        - 0.5 * ((sample - mu) / sigma_safe) ** 2
    )
    return torch.where(mask, torch.zeros_like(log_prob), log_prob)
```

Flow-SDE 分支的均值权重可以直接化简。令 u_i 为当前递减时间，$\Delta_i = u_i - u_{i+1}$，并记

$$\widehat{x}_0 = x_i - v_i u_i, \qquad \widehat{x}_1 = x_i + v_i(1 - u_i),$$

则官方代码对应

$$\mu_i = \widehat{x}_0[1 - (u_i - \Delta_i)] + \widehat{x}_1 \left[u_i - \Delta_i - \frac{\sigma_i^2 \Delta_i}{2u_i} \right].$$

展开后为

$$\mu_i = x_i - v_i \Delta_i - \frac{\sigma_i^2 \Delta_i}{2u_i} \left[x_i + (1 - u_i) v_i \right].$$

第一项是递减时间约定下的确定性 Euler 更新，第二项是 Flow-SDE 的漂移修正。标准差由 $\sigma_i^{\text{step}} = \sigma_i \sqrt{\Delta_i}$ 给出。

这里有两个实现细节。第一，`sample_mean_var` 返回的是 (μ, σ)，第二个量是标准差而不是方差；采样写成 $x_{i+1} = \mu_i + \sigma_i \epsilon$。第二，`gaussian_logprob` 计算逐元素对数概率，并在标准差为零的确定性坐标上返回零。动作块维度的裁切与聚合发生在上层策略代码中，而不是该函数内部。

1.7 官方 Rollout：随机转移如何进入训练数据

`Pi0RL._predict_train` 在每个去噪步骤调用速度模型得到 v_t，按所选方法计算转移均值和标准差，再采样下一状态、保存轨迹链及旧策略对数概率。若只需一个随机去噪决策，其他步骤会走 `flow_ode`；联合轨迹概率配置则会对所有去噪步骤记录概率。该实现细节解释了论文中的混合 ODE–SDE rollout 如何减少显式随机决策的数量。

Listing 1.2: RLinf 官方 `Pi0RL._predict_train` 关键片段：选择转移类型、采样并记录旧策略对数概率。

```
# Copyright 2026 The RLinf Authors.
# Licensed under the Apache License, Version 2.0.
# Source: rlinf/models/embodiment/openpi/tasks/rl.py, Pi0RL._predict_train
# https://github.com/RLinf/RLinf/blob/main/rlinf/models/embodiment/openpi/tasks/rl.py
#
# Curated excerpt of the official rollout loop. Surrounding observation,
# prefix-cache, value-head and NFT bookkeeping code is omitted.

for idx in range(num_steps):
    method = (
        rl_cfg.noise_method
        if idx == int(denoise_inds[0, idx].item())
        else "flow_ode"
    )

    t_val = float(timesteps[idx].item())
    t_tensor = torch.full((B,), t_val, device=device, dtype=torch.float32)
    suffix_act = self.run_suffix(observation, x_t, t_tensor, kv_cache, prefix_mask)
    v_t = self.velocity_from_suffix(suffix_act).to(torch.float32)

    noise_std = None
    if method == "flow_noise":
        noise_std = self.noise_head(suffix_act).to(torch.float32)

    idx_step.fill_(idx)
    x_t_mean, x_t_std = rl_sampler.sample_mean_var(
        x_t.to(torch.float32),
```

```
        v_t,
        idx_step,
        noise_method=method,
        noise_level=rl_cfg.noise_level,
        num_steps=num_steps,
        noise_std=noise_std,
    )

    step_noise = torch.randn(
        x_t.shape, device=device, dtype=torch.float32, generator=rng
    )
    x_t = x_t_mean + step_noise * x_t_std
    log_probs.append(rl_sampler.gaussian_logprob(x_t, x_t_mean, x_t_std))
    chains.append(x_t)
```

对应配置示例如下。Flow-SDE 使用预设噪声强度，并按转移级概率组织 rollout；Flow-Noise 则配置可学习噪声网络，并使用联合去噪轨迹概率。具体实验还会设置模型路径、动作块长度、训练步数和 Critic 结构，这里只保留决定两种噪声机制的字段。

Listing 1.3: RLinf 官方 Flow-SDE 配置节选。

```
# Excerpt from RLinf official pi0 / pi0.5 configuration documentation.
# Source: https://github.com/RLinf/RLinf/blob/main/docs/source-en/rst_source/
    ↪ examples/embodied/pi0.rst
openpi:
  noise_method: "flow_sde"
  noise_level: 0.5
  joint_logprob: False  # official guidance: Flow-SDE uses transition-level log-prob
```

Listing 1.4: RLinf 官方 Flow-Noise 配置节选。

```
# Excerpt from RLinf official pi0 / pi0.5 configuration documentation.
# Source: https://github.com/RLinf/RLinf/blob/main/docs/source-en/rst_source/
    ↪ examples/embodied/pi0.rst
openpi:
  noise_method: "flow_noise"
  noise_logvar_range: [0.08, 0.16]
  joint_logprob: True
```

训练时，rollout 记录的状态链、去噪索引和旧策略对数概率会随样本一起保存。推理时，官方策略进入 eval 路径，转移标准差置零，动作由确定性去噪过程产生；这与训练时为 PPO 提供概率的随机转移相区分。

第 2 章

Flow-Noise：通过学习噪声构造随机策略

2.1 从确定性 Euler 更新到高斯转移

Flow-Noise 保留 Flow-ODE 的均值更新，并让可学习噪声网络预测逐元素标准差。论文用递增时间 τ 说明其基本思想；RLinf 当前官方实现采用递减时间网格 $u_i = 1 - i/K$，因此源码中的 Euler 更新写成 $x_{i+1} = x_i - v_i \Delta_i$，其中 $\Delta_i = u_i - u_{i+1} > 0$。在官方采样器中，这一均值由两个端点预测的线性组合得到：

$$\widehat{x}_0 = x_i - v_i u_i, \qquad \widehat{x}_1 = x_i + v_i (1 - u_i),$$

$$\mu_i = \widehat{x}_0 [1 - (u_i - \Delta_i)] + \widehat{x}_1 (u_i - \Delta_i) = x_i - v_i \Delta_i.$$

噪声网络输入是 Action Expert 的隐藏特征 h_i，而不是直接把观测和动作拼接后输入一个新网络。其输出为与动作状态同形状的标准差张量：

$$\boldsymbol{\sigma}_{\theta'} = \text{ExploreNoiseNet}(h_i), \qquad x_{i+1} = \mu_i + \boldsymbol{\sigma}_{\theta'} \odot \boldsymbol{\epsilon}.$$

因此单步转移为对角高斯分布。该方法不使用 Flow-SDE 的 Score 漂移修正；随机性由可学习噪声网络控制。

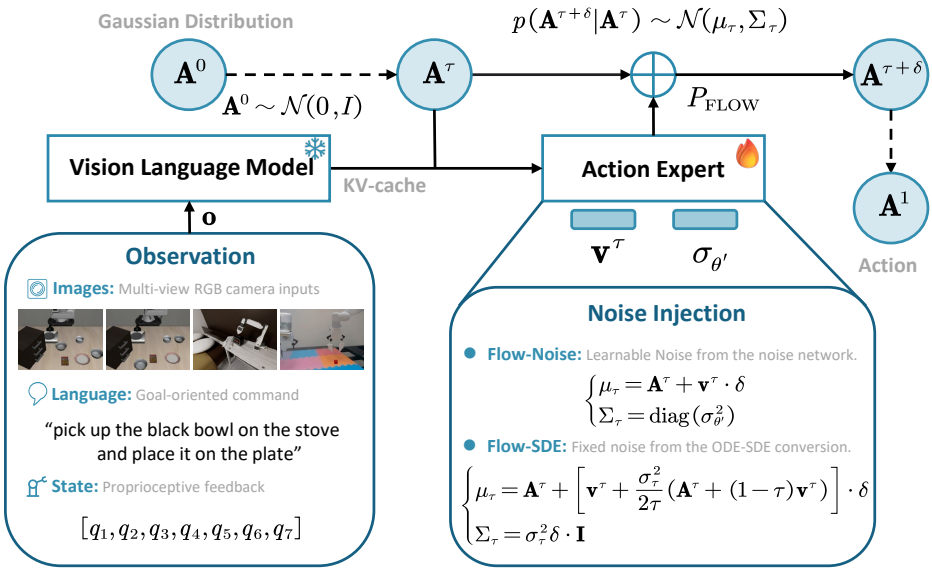


图 2.1: Flow-Noise 在原始 Flow 去噪步骤的均值更新上加入可学习的随机噪声。

论文中的协方差写为 $\text{diag}(\boldsymbol{\sigma}_{\theta'}^2)$，其具体缩放由噪声网络参数化决定。不能未经核对就将它替换为 Flow-SDE 的 $\sigma_\tau^2 \delta \mathbf{I}$：两种方法的噪声定义并不相同。

2.2 为什么整条去噪轨迹的对数概率可计算

设去噪过程被离散为 K 步。按论文符号，τ 从 0 增加到 1；按 RLinf 当前官方实现，时间变量 u_i 从 1 递减到 0，初始状态是噪声，最终状态是动作。以源码的记号表示，完整去噪链为

$$\mathcal{X} = (x_0, x_1, \ldots, x_K), \qquad u_0 = 1,\ u_K = 0.$$

利用概率的链式法则，理论上的轨迹联合对数概率为

$$\log p(\mathcal{X} \mid \mathbf{o}) = \log p(x_0) + \sum_{i=0}^{K-1} \log p(x_{i+1} \mid x_i, \mathbf{o}).$$

初始状态 x_0 来自标准高斯分布；每个后续状态来自可计算的高斯转移。这里的“可精确计算”指离散转移分布的概率密度已知，不意味着它等同于原始确定性 ODE 的最终动作边缘似然。

在 Flow-Noise 的单层 MDP 中，策略在每个去噪时刻选择下一个动作状态。去噪链上的概率比可以直接进入策略优化目标，因此不必为最终动作求解难以处理的连续流似然。

2.3 官方噪声网络与转移计算

Flow-Noise 配置下才创建 `ExploreNoiseNet`。网络以 Action Expert 隐藏维度作为输入维度，以动作维度作为输出维度，隐藏层为 128 和 64 个单元，使用 Tanh 激活；噪声范围由配置提供。

Listing 2.1: RLinf 官方 `Pi0RL` 中 Flow-Noise 噪声网络的构造代码节选。

```
# Copyright 2026 The RLinf Authors.
# Licensed under the Apache License, Version 2.0.
# Source: rlinf/models/embodiment/openpi/tasks/rl.py, Pi0RL.__init__
# https://github.com/RLinf/RLinf/blob/main/rlinf/models/embodiment/openpi/tasks/rl.py
#
# Curated excerpt: official Flow-Noise head construction.

if rl_cfg.noise_method == "flow_noise":
    from rlinf.models.embodiment.modules.explore_noise_net import ExploreNoiseNet

    self.noise_head = ExploreNoiseNet(
        in_dim=action_expert_width,
        out_dim=self.model_action_dim,
        hidden_dims=[128, 64],
        activation_type="tanh",
        noise_logvar_range=list(rl_cfg.noise_logvar_range),
        noise_scheduler_type="learn",
    )
```

噪声网络将 MLP 输出限制到配置的 log-variance 区间，再转换为标准差。官方代码的关键变换如下：

Listing 2.2: RLinf 官方 `ExploreNoiseNet` 的噪声范围映射与前向计算节选。

```
# Copyright 2025 The RLinf Authors.
```

```
# Licensed under the Apache License, Version 2.0.
# Source: rlinf/models/embodiment/modules/explore_noise_net.py
# https://github.com/RLinf/RLinf/blob/main/rlinf/models/embodiment/modules/explore_noise_net.py
#
# Curated excerpt: ExploreNoiseNet methods used by Flow-Noise. The MLP helper
# is defined in the same official source file but omitted from this excerpt.

class ExploreNoiseNet(nn.Module):
    """Neural network to generate learnable exploration noise."""

    def __init__(
        self,
        in_dim: int,
        out_dim: int,
        hidden_dims: list[int],
        activation_type: str,
        noise_logvar_range: list,
        noise_scheduler_type: str,
    ):
        super().__init__()
        self.mlp_logvar = MLP(
            [in_dim] + hidden_dims + [out_dim],
            activation_type=activation_type,
            out_activation_type="Identity",
        )
        self.noise_scheduler_type = noise_scheduler_type
        self.set_noise_range(noise_logvar_range)

    def set_noise_range(self, noise_logvar_range: list):
        self.noise_logvar_range = noise_logvar_range
        noise_logvar_min = self.noise_logvar_range[0]
        noise_logvar_max = self.noise_logvar_range[1]
        self.register_buffer(
            "logvar_min",
            torch.log(torch.tensor(noise_logvar_min**2, dtype=torch.float32)).unsqueeze(0),
        )
        self.register_buffer(
            "logvar_max",
```

```
            torch.log(torch.tensor(noise_logvar_max**2, dtype=torch.float32)).
    ↪ unsqueeze(0),
        )

    def forward(self, noise_feature: torch.Tensor):
        if "const" in self.noise_scheduler_type:
            noise_std = torch.exp(0.5 * self.logvar_min)
        else:
            noise_logvar = self.mlp_logvar(noise_feature)
            noise_std = self.post_process(noise_logvar)
        return noise_std

    def post_process(self, noise_logvar):
        """Map network outputs into the configured log-variance range."""
        noise_logvar = torch.tanh(noise_logvar)
        noise_logvar = (
            self.logvar_min
            + (self.logvar_max - self.logvar_min) * (noise_logvar + 1) / 2.0
        )
        noise_std = torch.exp(0.5 * noise_logvar)
        return noise_std
```

源码中 noise_logvar_range 保存的是允许的标准差上下界；代码先将边界平方后取对数，网络输出经 tanh 映射到该 log-variance 区间，最后通过 $\exp(\frac{1}{2}\log\sigma^2)$ 得到标准差。这样可以避免网络输出任意大的噪声尺度。

官方 rollout 将 suffix_act 输入 noise_head 得到标准差，再作为 noise_std 传给采样器的 sample_mean_var，随后记录高斯转移概率。因此，噪声网络的训练不是独立的监督学习任务；它通过强化学习目标参与策略更新。

2.4 Flow-Noise 的训练与推理

训练时，速度网络与噪声网络共同参与策略优化。速度网络决定动作生成轨迹的方向，噪声网络决定每一步随机转移的尺度。每次 rollout 记录高斯转移的对数概率，机器人执行最终动作块后得到奖励，再由 PPO 更新可训练参数。论文指出，噪声网络在 RL 微调后会被丢弃，推理时保留速度网络并恢复确定性 ODE 采样。

这一安排将训练期和部署期的职责分开：训练期需要随机转移来支持探索与概率比计算；部署期则不必保留专门预测探索噪声的网络。需要注意，RL 更新已经改变速度网络参数，因此最终确定性策略的生成分布不必与初始 SFT 策略完全相同。

一次 Flow-Noise rollout 的计算顺序可以概括为：固定环境观测；从高斯噪声初始化动作状态；在每个去噪步预测速度和噪声尺度、采样下一状态并记录其对数概率；将最终动作块交给环境执行；将奖励与轨迹数据交给 PPO。优化阶段根据旧策略记录的对数概率和新策略重新计算的对数概率构造概率比，并通过策略梯度同时调整速度网络和噪声网络。噪声网络只服务于训练期的随机转移，完成 RL 微调后不进入确定性 ODE 推理路径。

第 3 章

PPO 与 VLA 强化学习训练

3.1 环境交互、动作块与奖励

将机器人任务写成 MDP：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P_0, P_{\text{ENV}}, R_{\text{ENV}}, \gamma).$$

状态 s_t 对应机器人观测 $\mathbf{o}_t$，动作是 VLA 输出的动作块 $\mathbf{A}_t$。环境根据动作块改变状态，并返回奖励。优化目标为最大化折扣累计回报：

$$\mathcal{J}(\pi_\theta) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t R_t\right].$$

π 系列模型一次输出未来 H 个控制时刻的动作。论文将一个动作块作为单个宏动作，其奖励为块内奖励之和：

$$R_t = \sum_{j=0}^{H-1} r_{t,j}.$$

这种定义将 Flow 去噪过程与真实机器人控制分开：τ 只索引动作块内部的生成过程，t 才推进环境状态。对于 Flow-SDE 的两层 MDP，内部去噪转移的奖励为零，只有最终动作块与环境交互后才获得环境奖励。

3.2 从对数概率到 PPO 概率比

策略梯度依赖动作对数概率：

$$\nabla_\theta \mathcal{J}(\pi_\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) A(s_t, a_t)\right].$$

对 Flow-Noise，概率可以取整条离散去噪轨迹的联合对数概率；对 Flow-SDE，可以使用两层 MDP 中被策略采样的高斯转移概率。两种方法提供了不同的概率建模方式，但最终都将概率比用于 PPO 更新。

令旧策略参数为 θ_{old}，新策略参数为 θ，则

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

在去噪 MDP 中，相应地将 a_t, s_t 替换为内部状态和转移动作 $\bar{a}_t^\tau, \bar{s}_t^\tau$。PPO 的裁剪目标为

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_t\big[\min\{\rho_t(\theta)\hat{A}_t, \\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\}\big].$$

当概率比偏离 1 过多时，裁剪项限制优势带来的更新幅度，减少单次策略更新过大的风险。

3.3 Critic 与 GAE 优势估计

Critic 估计状态价值 $V(s_t)$。利用当前块奖励和相邻状态的价值预测，可以构造 TD 误差：

$$\mathcal{T}_t = R_t + \gamma V(s_{t+1}) - V(s_t).$$

GAE 将多个未来 TD 误差加权累加：

$$\hat{A}_t = \sum_{k=0}^{T-t} (\gamma\lambda)^k \mathcal{T}_{t+k}.$$

γ 控制未来奖励的折扣程度，λ 控制优势估计的偏差与方差权衡。优势为正表示该动作优于当前价值基线，策略更新倾向于提高其概率；优势为负则倾向于降低其概率。

Critic 的连接位置依赖 VLA 架构。论文对 $\pi_{0.5}$ 将 Critic 接在 VLM 输出后，直接根据图像、语言和状态特征估计价值。对于 π_0，由于 Action Expert 的输入同时包含带噪动作与状态，论文通过沿去噪轨迹的价值预测近似观测价值：

$$V_{\text{expert}}(\mathbf{o}_t) \approx \mathbb{E}_{\tau\sim U[0,1]}\left[V_{\text{expert}}(\mathbf{o}_t, \mathbf{A}_t^\tau)\right].$$

这一区别来自两个模型的特征组织方式，而不是 Flow-Noise 与 Flow-SDE 的本质差异。

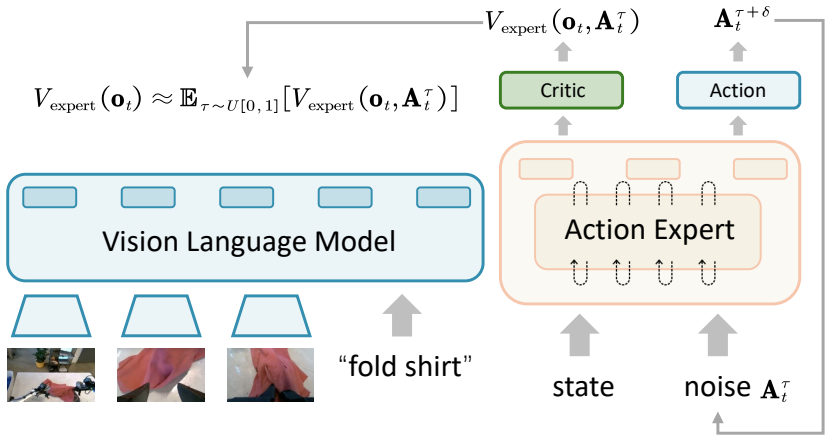


图 3.1: π_0 中 Critic 与 Action Expert 的连接方式。

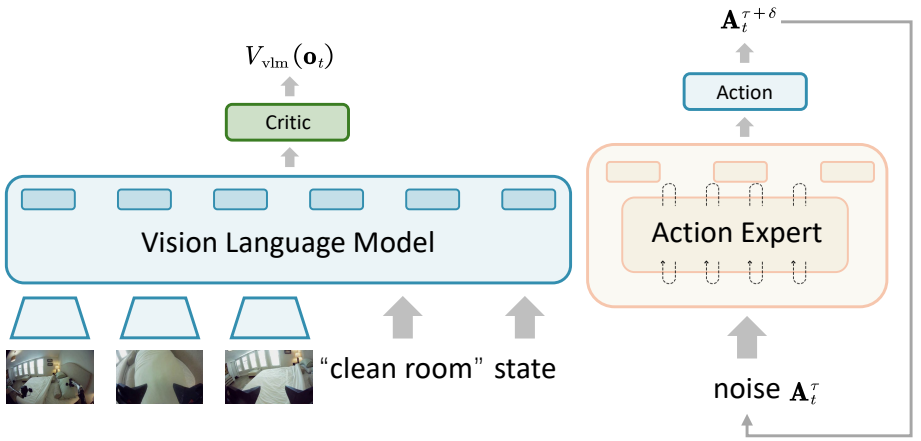


图 3.2: $\pi_{0.5}$ 中 Critic 连接在 VLM 输出后的结构。

3.4 在线训练循环与参数更新

RLinf 的模型侧代码负责根据 rollout 中保存的去噪链，使用当前参数重新计算策略对已采样转移的对数概率与价值估计。通用 PPO/GRPO 训练层再将这些输出与旧策略概率、优势估计和奖励结合，计算优化目标。为避免把框架外的伪代码误当作 πRL 官方实现，本节只列出模型侧真实代码路径中的概率重算逻辑。

Listing 3.1: RLinf 官方 `Pi0RL.default_forward` 关键片段：重算当前策略的转移对数概率。

```python
# Copyright 2026 The RLinf Authors.
# Licensed under the Apache License, Version 2.0.
# Source: rlinf/models/embodiment/openpi/tasks/rl.py, Pi0RL.default_forward
# https://github.com/RLinf/RLinf/blob/main/rlinf/models/embodiment/openpi/tasks/rl.py
#
# Curated excerpt: recompute probabilities for recorded denoising transitions
# under the current policy during PPO/GRPO optimization.

for idx in range(num_recompute):
    denoise_ind = denoise_inds[:, idx].to(torch.long)
    chains_pre = chains[arange_B, denoise_ind]
    chains_next = chains[arange_B, denoise_ind + 1]
    t_input = timesteps[denoise_ind].to(torch.float32)

    suffix_act = self.run_suffix(
        observation, chains_pre, t_input, kv_cache, prefix_mask
    )
    v_t = self.velocity_from_suffix(suffix_act).to(torch.float32)

    noise_std = None
    if rl_cfg.noise_method == "flow_noise":
        noise_std = self.noise_head(suffix_act).to(torch.float32)

    x_t_mean, x_t_std = rl_sampler.sample_mean_var(
        chains_pre.to(torch.float32),
        v_t,
        denoise_ind,
        noise_method=rl_cfg.noise_method,
        noise_level=rl_cfg.noise_level,
        num_steps=self.num_steps,
        noise_std=noise_std,
    )
    step_logprobs.append(
        rl_sampler.gaussian_logprob(chains_next.to(torch.float32), x_t_mean, x_t_std)
    )
```

```
36
37 # Output shaping in the same official method (surrounding value-head logic omitted).
38 log_probs = torch.stack(step_logprobs, dim=1)
39 log_probs = log_probs[:, :, : self.action_chunk, : self.action_env_dim].float()
40 if rl_cfg.joint_logprob:
41     log_probs = log_probs.mean(dim=1)
42 else:
43     log_probs = log_probs[:, 0]
44
45 # The official method returns policy statistics to the PPO/GRPO training layer.
46 return {
47     "logprobs": log_probs.contiguous(),
48     "values": values.float(),
49     "entropy": entropy.float(),
50 }
```

该代码从缓存的 chains 读取当前去噪状态和下一状态，重新运行 Action Expert，计算当前速度；Flow-Noise 分支还重新计算噪声标准差。然后，采样器用当前策略给出的 (μ_i, σ_i) 评估 rollout 中实际采样到的下一状态，得到新策略对数概率。PPO 训练层用新旧 log-prob 的差计算概率比：

$$\rho_i(\theta) = \exp\big(\log p_\theta(x_{i+1} \mid x_i, o) - \log p_{\theta_{\text{old}}}(x_{i+1} \mid x_i, o)\big).$$

Flow-Noise 的 `joint_logprob=True` 会在 rollout 与重算阶段保留初始噪声及各去噪步骤的 log-prob。需要区分理论定义与当前代码的归约：数学上的联合 log-prob 是各项相加；当前 RLinf `Pi0RL` 实现将去噪步维度的 log-prob 张量用 `mean(dim=1)` 归约，再交给 PPO/GRPO 层。因此，源码返回值是按步平均后的策略统计，而不是未经归约的联合 log-prob 总和。Flow-SDE 的常用配置则将随机决策集中在选定的去噪步骤，按对应转移概率进行优化。

RLinf 官方配置明确区分两种方法的开关：

- Flow-SDE：`noise_method: flow_sde`，由固定 `noise_level` 控制噪声强度，常用 `joint_logprob: False`。
- Flow-Noise：`noise_method: flow_noise`，构造可学习 `ExploreNoiseNet`，常用 `joint_logprob: True`。

这些配置字段来自 RLinf 的官方 π0/π0.5 说明及实验 YAML。Action Expert 是否单独训练、Critic 接在 VLM 还是 Action Expert 后面，由同一配置体系控制；具体设置应与所使用的模型版本和实验配置一致。

3.5 推理：从观测到机器人动作

训练阶段的随机转移用于探索与 PPO 概率计算；推理阶段不计算优势，也不更新参数。系统读取当前相机图像、任务指令和机器人状态，采样初始高斯噪声。论文的递增时间写法是从 $\tau = 0$ 积分到 $\tau = 1$；RLinf 当前官方实现则用 u_i 从 1 递减到 0，在 `flow_ode` 分支中将确定性均值更新为

$$x_{i+1} = x_i - v_i \Delta_i, \qquad \Delta_i = u_i - u_{i+1} > 0.$$

最终的 x_K 是动作块。机器人执行动作块中规定的控制动作后，系统获取新观测，再生成下一段动作块。因而部署是闭环过程：观测、生成动作、执行、重新观测不断循环。

训练后使用确定性 ODE，并不意味着 RL 微调前后动作分布必然相同。PPO 已经更新了速度网络参数；确定性描述的是给定观测和初始噪声后的去噪更新方式，而不是说初始噪声或最终动作在不同调用间必须相同。

参考资料

1. Chen, K. et al. *π_{RL}: Online RL Fine-tuning for Flow-based Vision-Language-Action Models.* arXiv:2510.25889, 2025. 论文页面.
2. RLinf Authors. *RLinf: Reinforcement Learning Infrastructure for Embodied and Agentic AI.* 官方仓库：github.com/RLinf/RLinf（本书核对 `main` 分支，2026-10-29）。
3. RLinf Authors. `rlinf/models/embodiment/openpi/sampling/rl_sampler.py`. 官方采样器源码.
4. RLinf Authors. `rlinf/models/embodiment/openpi/tasks/rl.py`. 官方 Pi0RL 实现.
5. RLinf Authors. `rlinf/models/embodiment/modules/explore_noise_net.py`. 官方噪声网络实现.
6. Song, Y. et al. *Score-Based Generative Modeling through Stochastic Differential Equations.* ICLR, 2021.
7. Schulman, J. et al. *Proximal Policy Optimization Algorithms.* arXiv:1707.06347, 2017.
8. Lipman, Y. et al. *Flow Matching for Generative Modeling.* ICLR, 2023.